

PRD — Classroom BLE Mesh Attendance System

Basic Product Requirements Document / AI Coding-Agent Handoff

Document type: Basic Product Requirements Document / AI Coding-Agent Handoff

Status: Draft MVP

Primary hardware: ESP32-S3

Primary networking concept: BitChat-inspired application-level BLE peer mesh

Attendance trigger: One-time session at the end of a lecture

Core verification: BLE presence + face verification

1. Product Summary

Build a classroom attendance system in which students open a mobile app at the end of a lecture and tap **Mark Attendance**. The app temporarily activates the device's BLE networking functions, discovers nearby participating phones, and can relay attendance packets through nearby student phones using a BitChat-inspired application-level BLE mesh.

An **ESP32-S3** placed in the classroom acts as a hardware gateway. It receives attendance packets over BLE and forwards them to the backend over Wi-Fi.

Face verification is used to confirm that the person submitting attendance matches the enrolled student identity.

Target flow

```
text
Lecture ends
↓
Teacher starts attendance session
↓
Students open app
↓
Student taps "Mark Attendance"
↓
Face verification
↓
Phone joins temporary BLE mesh
↓
Attendance packet is relayed through nearby phones
↓
ESP32-S3 gateway receives packet
↓
ESP32 → Wi-Fi → Backend
↓
Server validates session/token
↓
Attendance recorded
```

2. Problem

Conventional fingerprint attendance requires students to physically visit a scanner and can create queues. A classroom-wide digital system should allow students to submit attendance simultaneously while providing stronger evidence than a simple button press.

This project explores a contactless, classroom-local approach combining:

- peer-to-peer BLE communication,
- multi-hop application-level forwarding,
- an ESP32 hardware gateway,
- cryptographically protected attendance tokens,
- and face verification.

This project is **not** intended to claim that BLE mesh is universally better than fingerprint attendance. Its value is simultaneous classroom collection and experimentation with decentralized wireless networking.

3. Goals

MVP goals

1. Allow a teacher to create a short-lived attendance session.
2. Allow students to open the app and submit attendance once.
3. Perform face verification or a clearly defined identity-verification placeholder.
4. Create a short-lived signed/cryptographically protected attendance token.
5. Send the token using BLE.
6. Support direct phone → ESP32 communication.
7. Add phone → phone → ESP32 multi-hop forwarding as the main experimental mesh feature.
8. Have the ESP32 forward valid packets to a backend over Wi-Fi.
9. Prevent duplicate packet processing.
10. Display attendance status to the teacher.

Later goals

- Android + iOS interoperability.
- Multi-hop reliability for 20+ devices.
- Better anti-replay protection.
- Multiple gateway support.
- RSSI/proximity evidence.
- Detailed diagnostics and mesh visualization.

— — —

4. Non-Goals

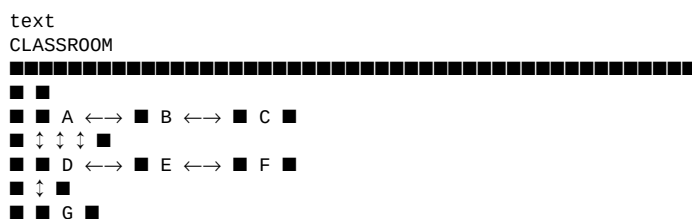
The MVP does **not** attempt to:

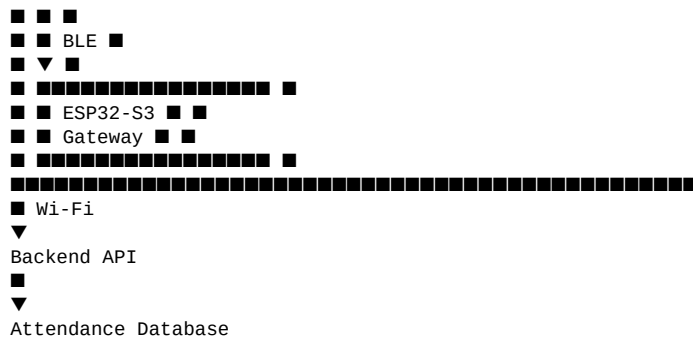
- implement Bluetooth SIG Mesh on student phones;
- keep a mesh running continuously during the lecture;
- guarantee an exact physical radius such as 20 feet;
- replace institutional biometric systems;
- perform unrestricted background BLE networking on iOS;
- make face recognition run on the ESP32;
- build a production-grade biometric identity platform;
- guarantee attendance solely from Bluetooth proximity.

The app is intentionally opened at the end of the lecture, which reduces the importance of continuous background BLE operation.

— — —

5. Architecture





Important terminology

This project uses a **BitChat-inspired application-level BLE mesh**.

It should not be described as the same thing as **Bluetooth SIG Mesh**.

ESP32's official ESP-BLE-MESH stack is useful for learning and for alternative experiments, but the student-phone mesh is an application-level peer relay design.

6. Components

6.1 Student mobile app

Responsibilities:

- student authentication/enrollment;
- attendance session discovery;
- face capture/verification;
- BLE scanning;
- BLE advertising;
- peer discovery;
- packet forwarding;
- duplicate suppression;
- TTL/hop handling;
- secure packet creation;
- attendance status.

6.2 ESP32-S3 gateway

Responsibilities:

- advertise/discover the classroom gateway service;
- receive attendance packets over BLE;
- validate basic packet structure;
- suppress obvious duplicates;
- optionally acknowledge receipt;
- send received packets to backend over Wi-Fi;
- provide diagnostic information.

The ESP32 should **not** perform full face recognition.

6.3 Backend

Responsibilities:

- teacher/session management;
- student enrollment;
- token/session validation;
- attendance database;
- duplicate prevention;
- audit records;
- teacher dashboard.

7. Proposed Attendance Protocol

A conceptual packet can contain:

```
json
{
  "version": 1,
  "type": "ATTENDANCE",
  "packetId": "random-unique-id",
  "sessionId": "short-lived-session-id",
  "studentToken": "temporary-token",
  "timestamp": 0,
  "ttl": 5,
  "hopCount": 0,
  "signature": "digital-signature"
}
```

Rules

- `packetId` uniquely identifies a submission.
- `sessionId` limits attendance to the active lecture.
- `studentToken` should be temporary and should not expose a permanent student identifier over BLE.
- `timestamp` helps limit stale/replayed packets.
- `ttl` prevents packets from circulating indefinitely.
- `hopCount` records forwarding depth for diagnostics.
- `signature` authenticates the submission.
- Phones keep a short-lived cache of recently seen packet IDs.

The final field layout should be decided after the first BLE prototype.

8. Mesh Forwarding

The basic forwarding algorithm:

```
text
Receive packet
↓
Is packet ID already seen?
■■■ YES → discard
■■■ NO
↓
Store packet ID temporarily
↓
Validate session + TTL
↓
If gateway is reachable:
send toward gateway
↓
Otherwise:
decrement TTL
↓
forward to selected peers
```

The MVP should avoid uncontrolled flooding. Start with a simple bounded forwarding policy.

9. Face Verification

Recommended flow:

```
text
Tap Mark Attendance
↓
Capture face
↓
Verify against enrolled identity
↓
Generate attendance token
↓
Send token through BLE mesh
```

Face processing should initially run on the phone or backend, not the ESP32.

The system should store the minimum biometric information required by the chosen implementation and should not transmit raw face images through the mesh.

10. Teacher Flow

1. Teacher opens dashboard.
2. Teacher selects class.
3. Teacher starts attendance.
4. Backend generates a short-lived `sessionId`.
5. ESP32 gateway associates with that session.
6. Students submit attendance.
7. Dashboard shows:
 - Present
 - Pending
 - Failed verification
 - Duplicate/rejected
8. Teacher closes the session.

11. Student Flow

1. Student opens app after lecture.
2. App shows the active attendance session.
3. Student taps **Mark Attendance**.
4. App requests required permissions.
5. Face verification runs.
6. App generates a temporary attendance token.
7. App discovers nearby mesh peers/gateway.
8. Packet is sent directly or relayed.
9. App displays:
 - `Attendance submitted`
 - `Waiting for gateway`
 - or `Submission failed`

12. Security Requirements

The system must consider:

- replay attacks;
- copied student tokens;
- impersonation;
- duplicate submissions;
- forged attendance packets;
- stale sessions;
- unauthorized gateways;
- packet tampering.

Minimum MVP protections:

1. short-lived sessions;
2. random packet IDs;
3. timestamps/expiry;
4. cryptographic signing or MAC;
5. server-side validation;
6. temporary BLE identifiers;
7. duplicate suppression.

Do not assume that BLE proximity alone proves physical presence.

13. Privacy

The system should minimize personal information transmitted over BLE.

Prefer:

```
text
BLE:
temporary token + session + cryptographic proof
```

instead of:

```
text
BLE:
student name + roll number + phone number
```

Face images should not be broadcast through the mesh.

The final implementation must document what biometric information is stored, where it is stored, and how it is deleted.

14. iOS / Android Considerations

The first implementation should keep the attendance flow **foreground-only**.

Students explicitly open the app at the end of the lecture. This is intentional.

iOS Core Bluetooth has platform-specific background execution behavior and restrictions. Therefore, the MVP should not depend on an app silently running BLE mesh networking for an entire lecture.

Android and iOS should be tested separately before claiming full cross-platform reliability.

15. Reference Material

A. BitChat — PRIMARY architectural reference

Repository:

<https://github.com/permissionlesstech/bitchat>

Use it to study:

- peer discovery;
- BLE central/peripheral architecture;
- packet structures;
- TTL-based routing;
- duplicate handling;
- forwarding;
- fragmentation;
- cross-platform protocol concepts.

The current BitChat repository states that it is released into the public domain, but the AI agent must verify the exact license files and current repository state before copying code into this project.

BitChat's current protocol whitepaper describes an ad-hoc BLE mesh and store-and-forward architecture.

B. BitChat protocol implementation

Relevant file:

``bitchat/Protocols/BitchatProtocol.swift``

Useful concepts include:

- binary packet encoding;
- message types;
- TTL;
- fragmentation;
- relay-related protocol behavior.

Do **not** copy the BitChat attendance semantics. Adapt only appropriate networking concepts.

C. BitChat Android

<https://github.com/permissionlesstech/bitchat-android>

Useful for:

- Android BLE implementation;
- cross-platform protocol compatibility;
- peer discovery;
- multi-hop relay concepts.

D. BitChat ESP32 implementation

<https://github.com/hackerhouse-opensource/bitchat-esp32>

Useful as a reference for:

- ESP32-side BitChat-compatible BLE behavior;
- packet diagnostics;
- BLE gateway/peer experimentation.

Check the repository's current supported chip and license before using code.

E. ESP-IDF ESP32-S3 BLE documentation

<https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-guides/ble/overview.html>

Use for:

- ESP32-S3 BLE APIs;
- GATT/GAP;
- NimBLE;
- Bluedroid;
- official ESP32 BLE examples.

ESP32-S3 supports BLE and Espressif documents both Bluedroid and NimBLE options.

F. ESP-BLE-MESH documentation

Use official Espressif ESP-BLE-MESH documentation for learning:

- provisioning;
- relay;
- mesh nodes;
- BLE Mesh concepts.

This is a **different architecture** from the phone application-level mesh and should not be confused with it.

G. Apple Core Bluetooth

<https://developer.apple.com/documentation/corebluetooth>

Use for:

- iOS BLE central/peripheral APIs;
- scanning;
- advertising;
- connections;
- platform restrictions.

H. Android BLE

Use official Android Bluetooth/BLE documentation for:

- scanning;
- advertising;
- GATT;
- runtime permissions;
- Android-specific BLE behavior.

I. Supplied ESP32 BLE Mesh tutorial

Video supplied for this project:

<https://youtu.be/DbrN5ONz3O4?si=Fe1copWS3j64vPfE>

Use it as an educational reference for ESP32/ESP-IDF BLE Mesh concepts.

It should **not** be treated as the specification for the phone mesh.

16. What the AI Coding Agent May Reuse

Safe to use as learning/reference

- BitChat architecture and whitepaper.
- BitChat packet/relay concepts.
- BitChat source structure.
- BitChat Android BLE implementation patterns.

- Espressif BLE examples.
- ESP-IDF APIs.
- Apple Core Bluetooth APIs.
- Android BLE APIs.
- ESP32 BLE Mesh examples for comparison.

Potentially reusable code

BitChat's current repository is public-domain according to its repository/whitepaper materials. However, before copying code, the agent must:

1. inspect the exact license file in the checked-out commit;
2. preserve required attribution/notices if applicable;
3. record the upstream commit/version;
4. isolate imported code;
5. document modifications.

Must be implemented specifically for this project

- attendance session protocol;
- student enrollment;
- temporary attendance tokens;
- teacher session management;
- face-verification integration;
- attendance database;
- attendance-specific security rules;
- anti-replay logic;
- attendance UI;
- ESP32 attendance gateway behavior.

17. Recommended Repository Structure

```

text
attendance-mesh/
  ■■■ mobile/
  ■ ■■■ android/
  ■ ■■■ ios/
  ■
  ■■■ esp32-gateway/
  ■ ■■■ src/
  ■ ■■■ include/
  ■ ■■■ README.md
  ■
  ■■■ backend/
  ■ ■■■ api/
  ■ ■■■ auth/
  ■ ■■■ database/
  ■
  ■■■ protocol/
  ■ ■■■ README.md
  ■ ■■■ packet-spec.md
  ■ ■■■ test-vectors/
  ■
  ■■■ docs/
  ■ ■■■ architecture.md
  ■ ■■■ security.md
  ■ ■■■ references.md
  ■
  ■■■ tests/
  ■■■ protocol/
  ■■■ android/

```

■■■ ios/
■■■ esp32/

18. Development Phases

Phase 0 — Research

Read and understand:

- BitChat whitepaper;
- BitChat iOS/Android networking code;
- ESP-IDF BLE;
- Apple Core Bluetooth;
- Android BLE.

Phase 1 — ESP32 gateway

Build:

```
text
Phone → BLE → ESP32-S3 → Wi-Fi → Backend
```

No mesh yet.

Phase 2 — Phone-to-phone relay

Build:

```
text
Phone A → Phone B → ESP32
```

Start with Android devices.

Phase 3 — Multi-hop

Test:

```
text
A → B → C → ESP32
```

Add TTL and duplicate suppression.

Phase 4 — iOS interoperability

Test:

```
text
iPhone → Android → ESP32
iPhone → iPhone → ESP32
Android → iPhone → ESP32
```

Phase 5 — Attendance

Add:

- sessions;
- temporary tokens;
- face verification;
- backend validation.

Phase 6 — Scale testing

Test with:

- 3 devices;
- 5 devices;
- 10 devices;

- 20 devices;
- target classroom size.

Measure:

- delivery rate;
- latency;
- hop count;
- battery usage;
- duplicate rate;
- failure rate.

19. Acceptance Criteria

MVP

- ☐ ESP32-S3 receives BLE attendance packets.
- ☐ ESP32 sends packets to backend over Wi-Fi.
- ☐ Student can submit attendance from app.
- ☐ Attendance session expires.
- ☐ Duplicate submissions are rejected.
- ☐ Stale/replayed packets are rejected.
- ☐ Face verification is integrated or mocked behind a stable interface.
- ☐ Direct phone → ESP32 flow works reliably.

Mesh milestone

- ☐ Phone A can relay a packet through Phone B.
- ☐ Packet reaches ESP32 without direct A→ESP32 connectivity.
- ☐ Duplicate packets do not create duplicate attendance.
- ☐ TTL prevents indefinite forwarding.
- ☐ Mesh can recover when a peer disappears.

Final prototype target

- ☐ 10+ phones tested.
- ☐ Android-to-Android relay tested.
- ☐ At least one iOS interoperability path tested.
- ☐ Results documented rather than assumed.
- ☐ Limitations clearly reported.

20. Major Risks

Risk 1 — iOS BLE behavior

Mitigation: keep attendance foreground-only and prototype Android first.

Risk 2 — Mesh reliability

Mitigation: start with direct communication and incrementally add hops.

Risk 3 — Too many BLE connections

Mitigation: use advertisements/discovery and controlled connections rather than attempting a permanent connection to every student.

Risk 4 — False attendance

Mitigation: combine face verification, session-bound cryptographic tokens, and BLE evidence.

Risk 5 — Scope explosion

Mitigation: do not build the full BitChat feature set. Implement only the minimum networking functionality required for attendance.

21. Critical Engineering Principle

Do not begin by building the complete app.

The highest-risk technical question is:

> Can our selected Android/iOS BLE implementation reliably relay a small attendance packet across multiple phones and deliver it to an ESP32-S3 gateway while the attendance app is open?

Prove this first.

Recommended proof:

```
text
Phone A
↓ BLE
Phone B
↓ BLE
Phone C
↓ BLE
ESP32-S3
↓ Wi-Fi
Laptop/server
```

Only after this works should the team spend significant time on UI, face recognition, dashboards, and database features.

22. Open Questions

1. Native Android + native iOS or a cross-platform framework with native BLE modules?
2. Which face-verification implementation will be used?
3. Should the ESP32 gateway use NimBLE or another ESP-IDF BLE stack?
4. Should packets use a custom protocol inspired by BitChat or a deliberately smaller attendance-specific protocol?
5. How many simultaneous student devices must the MVP support?
6. How should the system behave if the mesh cannot reach the gateway?
7. What biometric data, if any, will be stored?
8. What exact hardware enclosure/power arrangement will be used for the classroom gateway?

23. Final Product Definition

The MVP is **not a BitChat clone**.

It is a **classroom attendance system inspired by BitChat's BLE peer-to-peer networking model**, using an ESP32-S3 as a physical gateway and face verification as an identity layer.

The implementation should reuse proven BLE networking ideas where legally and technically appropriate, while keeping the attendance protocol, backend, identity model, and security decisions specific to this project.

Priority order:

1. Prove BLE phone ↔ ESP32.
2. Prove phone ↔ phone relay.

3. Prove multi-hop.
4. Prove ESP32 gateway → backend.
5. Add attendance protocol/security.
6. Add face verification.
7. Add polished UI.
8. Scale-test and document limitations.